

How *Ron's Gone Wrong* Went Right With Procedurally Generated Vector Graphics

Ryan Chan
ryc@dneg.com
DNEG
London, UK

Jesus R. Nieto
crn@dneg.com
DNEG
London, UK

Toba Siebzener
tlsi@dneg.com
DNEG
London, UK

Paul Baaske
pajb@dneg.com
DNEG
London, UK

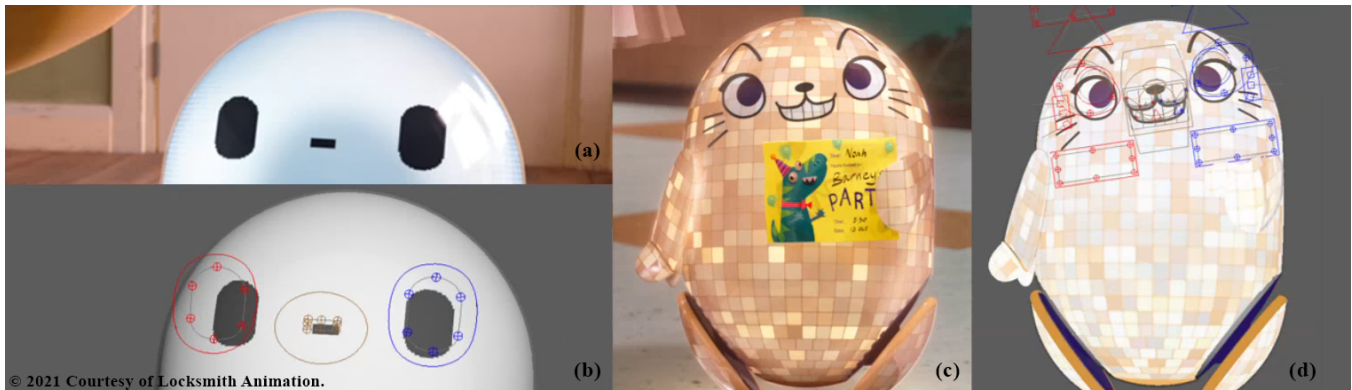


Figure 1: Rendering of Ron (a), with its vector graphics rig (b), and a more complex character's render (c) and rig (d).

ABSTRACT

In the feature animation film *Ron's Gone Wrong* there are many robot characters designed to have their surface working like a display. The acting of those characters relied mostly on moving graphic elements represented on their surface: animated faces, costumes and props. All graphic elements had to be art-directed, requiring the animation department to be able to control them interactively.

We propose a procedural workflow to produce animated textures from a rig, apply filters and project them onto the character's surface to mimic a screen-like effect. The workflow proved to be flexible and scalable to address many characters in production, with several levels of complexity and a lot of creative input.

CCS CONCEPTS

• Computing methodologies → Computer graphics; Animation.

KEYWORDS

Rigging, Faces, Costumes, Vector graphics

ACM Reference Format:

Ryan Chan, Jesus R. Nieto, Toba Siebzener, and Paul Baaske. 2022. How *Ron's Gone Wrong* Went Right With Procedurally Generated Vector Graphics. In *Special Interest Group on Computer Graphics and Interactive Techniques*

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH '22 Talks, August 07-11, 2022, Vancouver, BC, Canada

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9371-3/22/08.

<https://doi.org/10.1145/3532836.3536230>

Conference Talks (SIGGRAPH '22 Talks), August 07-11, 2022. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3532836.3536230>

1 INTRODUCTION

Locksmith Animation *Ron's gone wrong* presented a series of challenges for our character pipeline at DNEG. The robots' design included a capsule-like body, the surface of which works like a continuous screen that is extended to all modular arms and wheels to represent facial expressions and costumes. All of these had to be animated using a control rig to allow artists to achieve very elaborate compositions (see Figure 1). In addition to that, Ron (the main character) is malfunctioning, therefore he required extra controls and stylized effects adding an extra level of complexity.

The wide range of expressions of all the robots was a key element throughout the whole film, holding some of the most charming moments. This required real time feedback in order to efficiently direct the animation creative output.

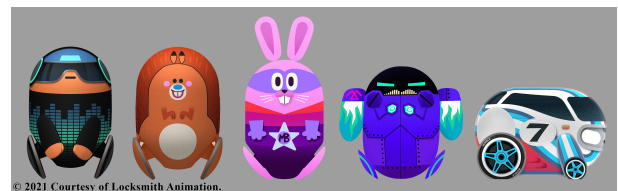


Figure 2: Examples of bots from the art department

2 PREVIOUS WORK

In the creation of Playmobil [Ringard and Levastre 2019], a similar problem was addressed to draw 2D animated faces. In this case,

the solution used a GPU-accelerated raycasting render of flat geometries executed in a separate viewport, solving occlusions and composition issues. Then the resulting procedurally-rendered image was applied (using UV coordinates) as a texture onto the surface of the face. Unfortunately, relying on UVs means that only very simple surfaces can be mapped without causing distortions or discontinuities at the UV seams, which was not the case for Ron's gone wrong. Moreover, as renderfarms typically don't provide support for GPUs, a CPU-friendly solution would be preferable.

Standard Autodesk Maya tools could also be used to project geometries on the robots' surfaces. Whenever complex compositions are needed (e.g. occlusions) boolean operations of the projected geometries could be useful. Unfortunately, such an approach quickly becomes very complex to manage and slow to compute, and causes precision issues when projecting on the surface, making it impractical.

Finally, a texture based approach could be used in some specific cases (e.g. wheels spinning or message notifications), but this is not suitable as a general solution as this would reduce artistic control.

3 PROCEDURAL TEXTURE GENERATION

To create the facial features and costume elements that we wanted to portray on the robots (see Figure 2) we decided to integrate vector graphics capabilities in our 3D pipeline. We developed a catalog of Maya nodes, which could be connected to create vector graphics in layers, with compositing operations, visibility, colors, ramps and transparency. Such nodes were integrated in our in-house framework [Nieto et al. 2018].

We relied on the Skia open source vector graphics library, developed by [Google 2021]. This provided most of the features we needed and it was ready for both GPU and CPU, which fits both our workstation and renderfarm hardware constraints.

The facial rig was based on spline curves rigged for facial animation. The curves were used to create vector shapes using Skia which were layered and grouped as needed. The contour, width, roundness, color and transparency parameters were made animatable for enhanced creative control. The composed vector graphics image was rasterised using Skia with two resolutions: low resolution for real time animation feedback and high resolution for the final rendering (see Figure 1).

To properly portray Ron's malfunctioning screen, the art department decided that resolution glitches and an old TV look could express the screen's malfunction. This was achieved by adding further nodes to our library: one to pixelate the image, keeping floating-point precision to avoid discontinuities in the animation, and another one to binarise and distort the image. Those filters were prototyped by one of our artists in Nuke and re-implemented as nodes in our network. Some parameters were exposed to animators to let them explore different glitch effects and contribute to the expressiveness of the character.

4 TEXTURE PROJECTION

To compute the ideal projection and minimize the typical distortion produced by standard methods (e.g. planar, cylindrical and spherical), we developed a custom solver that allowed us to establish a relation between the pixels on the image to be wrapped and the

vertices on the surface. A final step remaps the wrapped image onto a plane in a way that the texture can finally be projected using a standard Maya planar projection on the surface without distortions. This allowed us to have real-time visual feedback in the Maya and Katana viewports, without having to modify our UVs pipeline.

When projecting pixelated graphics, we had to find a way to prevent pixels from sliding over the surface when animating the projection. In order to keep the pixels snapped on the surface, we did counteract the movement of the projected plane as a floating-point offset on the image before the pixelation effect is applied.

5 THE SYSTEM IN PRODUCTION

With animation working closely with the Art, Motion Graphics and Rigging departments it was necessary to break down the stylistic requirements of each character. Each robot presented a different structure, determining the amount of layered graphics, the animation of these graphics and how animators control the setup. On a very complex character we created 37 controllable vector shapes of various colors and sizes, animated layered textures and a projected face which had to remain undistorted on the robot's capsule shape. At times the textures represented different facial expressions and needed to be directed by animation. Providing full vector rigs to animators proved to deliver excellent user experience as they could animate and see their work in real time, using a familiar interface.

With Ron's malfunctioning screen we also needed to be able to apply the glitching effects to the images that would be displayed on his body. At times we decided some things would be more efficiently resolved using standard Motion Graphics, without involving 3D animators. In these cases the animation department would receive a pre-rendered 2D animation to apply with the right timing on the character. Our solution allowed for the layering of multiple procedurally generated vector graphics and pre-rendered animated textures. This mixed workflow was powered by a new Texture Bundler system that allowed artists to assemble and manage all the elements for the various characters.

6 CONCLUSIONS AND FUTURE WORK

This solution proved to be very efficient and adaptable to different scenarios providing real-time animation with creative control and without integrating standard motion graphics workflows.

Workflows could be improved to import information from Photoshop or Illustrator files and translate them into rig-able vector graphics. This would allow us to streamline the translation of the 2D designs from the art department into animatable 3D rigs.

REFERENCES

- Google. 2021. Skia. <https://skia.org/docs/>.
- Jesus R Nieto, Charlie Banks, and Ryan Chan. 2018. Abstracting Rigging Concepts for a Future Proof Framework Design. In *Proceedings of the 8th Annual Digital Production Symposium* (Vancouver, British Columbia, Canada) (DigiPro '18). Association for Computing Machinery, New York, NY, USA, Article 3, 3 pages. <https://doi.org/10.1145/3233085.3233088>
- Jeremy Ringard and Claude Levastre. 2019. Facial Pipeline in Playmobil: The Movie. In *ACM SIGGRAPH 2019 Talks* (Los Angeles, California) (SIGGRAPH '19). Association for Computing Machinery, New York, NY, USA, Article 6, 2 pages. <https://doi.org/10.1145/3306307.3328207>